N-Watchmen Approximation

Eden Bendheim
Patrick Chen

Rensselaer Polytechnic Institute
CSCI 4650 - Computational Geometry

Abstract

Those interested in the field of computational geometry may be familiar with the famous art gallery problem, which asks: *In an art gallery, what is the minimum number of guards who together can observe the whole gallery?* Also of interest is the art gallery problem's popular extension, which is: *Where exactly should those minimal guards be placed to see the whole gallery?*

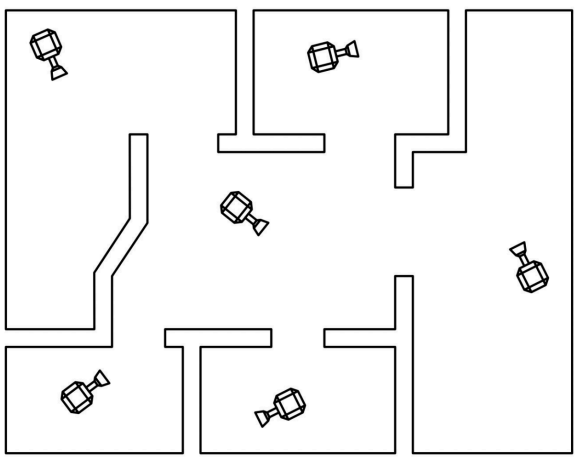

Figure 1: A simple polygonal shape being completely guarded by $n = 6$ stationary guards.

The topic this project is concerned with is a modification of the aforementioned problem(s). In a real-world scenario, there are certainly cases where a space must be guarded by stationary, 360-degree-seeing points. However, there are equally many, or even more, cases where a space must be guarded by "watchmen" who have the ability to move through the world. Furthermore, there are often cases when only a limited number of "cameras" are available, but they still must be used to completely watch some region. We study a version of the art gallery problem in which a fixed budget of n guards must be configured to see an entire area as effectively as possible. We call this the n-watchmen problem and seek to generate a good approximate solution for patrol over a simple polygonal space.

The n-watchmen problem is not only a theoretical exercise in computational geometry but useful for a variety of applications. For instance, space exploration programs may be interested in finding the best path for an extraterrestrial rover, armed with a radar and limited battery or engine life, to undertake on an alien planet. Urban or infrastructure designers may very well consult such tools when planning the large-scale layout of highway and railroad networks or telecommunications tower placement. This project, understanding that producing perfect solutions is an NP-hard problem, seeks to generate good approximations for these kinds of questions.

Problem Overview

We define the n-watchmen problem as such: *Given a simple polygonal area and a limited budget of exactly n guards, generate some patrolling configuration for the n guards that allow them to collectively see the entire area.*

Each "guard" can remain stationary at a single point or patrol infinitely along a connected polyline network. We do not consider the movement speed of the guards and allow them infinite time to complete their patrols. A "complete" patrol means the guard has at some time or another occupied every point along every line segment in their polyline network. This is an important clarification because patrols are polyline networks, not single polylines, so it may be necessary for a guard to "backtrack" through certain line segments to access unvisited

polylines in the network. Obviously, another constraint is that no patrol can leave the interior of the polygon. We allow patrol edges to be exactly collinear to a boundary edge of the polygon, which we can rationalize as a guard traveling "along the wall".

There are various variables that can be optimized for this problem. For one, we can try to minimize the total length of all the patrols undertaken by the guards. This optimization is useful for cases where the paths themselves are the main cost in the problem, such as if the problem concerns some infrastructure like roads, telephone lines, or gas pipes that emanate effects from points but must be connected over an area. Another possible optimization could be to minimize the *longest* patrol length; for example, to make sure no night guard patrolling a museum has an unfairly long assignment compared to their co-workers. For this project we are concerned with the first case.

Another constraint for our project is that we are not considering what a guard "sees" while moving. This is due to a technical limitation with generating visibility from entire line segments in addition to just from point locations, which we were unable to implement in time. As such, we will consider a patrol's visibility to be solely generated from the pivot points where they transition from one line segment to other(s).

Relevant Work

There is a large amount of related literature on the watchman and art gallery problems. During our research phase we focused on three references that were directly relevant to the multiple watchmen problem.

The foundational paper on this topic is Chin and Ntafos[1] 1991, which introduced the concept of the watchman problem and a polynomial time algorithm for computing the optimal solution for $n = 1$ in a simple, non-holey polygon. The presented algorithm has an $O(k^4 log(k))$ cost, with k being the size of the polygon. While a high polynomial runtime is not ideal, it establishes that the problem is solvable and not NP-hard for a basic case. Our project seeks to extend the solution for a general n, but it is less ambitious at the same time because we do not attempt to find the exact optimum. Regardless, the foundational paper is a valuable reference.

Tan[2] presented a linear time, 2-approximation algorithm for the $n = 1$ watchman route in simple polygons. This paper was important because it showed that good approximation guarantees are achievable. Our heuristics are very different compared to the techniques used by Tan, and does not provably produce a 2-approximation, but the paper gives us a target to compare against in spirit. Additionally, the paper provides various interesting test cases that were useful in the algorithm design process.

The third paper we examine is Packer[3] 2008, Computing Multiple Watchman Routes. Packer describes a similar problem to ours, where n watchmen must be assigned an optimal route configuration in a simple polygon. While we had only time to optimize for the sum of the total paths, Packer also considers an option for minimizing the longest single route. As with Tan's paper, we borrowed some test polygons from this paper, taking the largest inspiration from their set of examples.

Implementation

The algorithm code for this project is written in C++ with extensive use of the open-source Computational Geometry Algorithms Library (CGAL). Specifically, we employed the exact predicates, exact construction kernel (Epeck) for exact visibility computation. For visualization, we generate dynamic animations of the guards walking along their patrols using the Python library Manim.

I. Input and Output

A valid input to the algorithm consists of a constant value for n, the budget of guards, and a simple polygon in a text file formatted as points (x, y) in counter-clockwise order, with the last listed vertex implicitly forming an edge with the first vertex. A valid output consists of n patrols, where each patrol is either a set of connected edges or a single point. An output is considered valid if the union of the visibility polygons from

every point in every patrol is exactly equal to the original polygon. Otherwise, we say that the patrol configuration cannot be proven to guard the entire area, making it invalid. Additionally, since n can be an arbitrary number, it is possible for the guard budget to be "overkill" for a given area. We solve this by allowing guards to share the same patrol. The presence of duplicate patrols is a good indication that n guards is an inefficient allocation and fewer guards can just as easily see the entire area.

II. Algorithm Overview

Our algorithm consists of a base step to solve $n = 1$ and a reduction to create solutions for $n > 1$. Since we do not consider patrol edges when proving visibility, the reduction step can be accomplished easily by pruning edges, along with some other optimizations.

The solution for $n = 1$ employs a greedy incremental construction of a polyline network. We considered two methods for computing the solution; a simple method based on convexity properties, and a more advanced method using exact visibility polygons. Both methods track the region of the polygon that is unseen by the incremental solution, and both methods have approaches for converting an area from "unseen" to "seen". However, it is important to note that the unseen portion of the area (as tracked by the algorithm) is not exactly equal to the true unseen area by a guard navigating the incremental polyline network. It consists only of areas that can be proven to be seen by the current method.

III. Triangulation Method

Our first approach leveraged convexity principles and the polygon triangulation operation to generate working solutions. Since triangles are convex, we can say that a guard standing on any boundary of a triangle has a direct line of sight to every point in the triangle. Therefore, we can prove that the guard sees the entire area of the triangle. By marking all faces as initially belonging to the unseen area and using this proving method to subtract triangles from the unseen area, we can create a valid (if not optimal) solution.

To triangulate the polygon, we use the CGAL constrained triangulation package. However, the output of a constrained triangulation does not indicate which faces belong to the interior or exterior of the original polygon. To identify them, we perform a graph traversal of the triangulation, treating faces in the triangulation as nodes in the graph. Two nodes in the graph share an edge if and only if their associated faces share an edge in the triangulation. To mark every face as interior or exterior, we perform a graph traversal starting from the infinite face (which is always external). Whenever a constrained edge in the triangulation is crossed, the internality of the source face is reflected onto the target face; for example, if the traversal crosses a constrained edge from an internal face to some other face, the other face should be marked as external, as vice versa.

After identifying the internal faces, we convert the triangulation into a point-graph where the nodes are vertices of the original polygon and the edges are the internal edges formed between them by the triangulation.

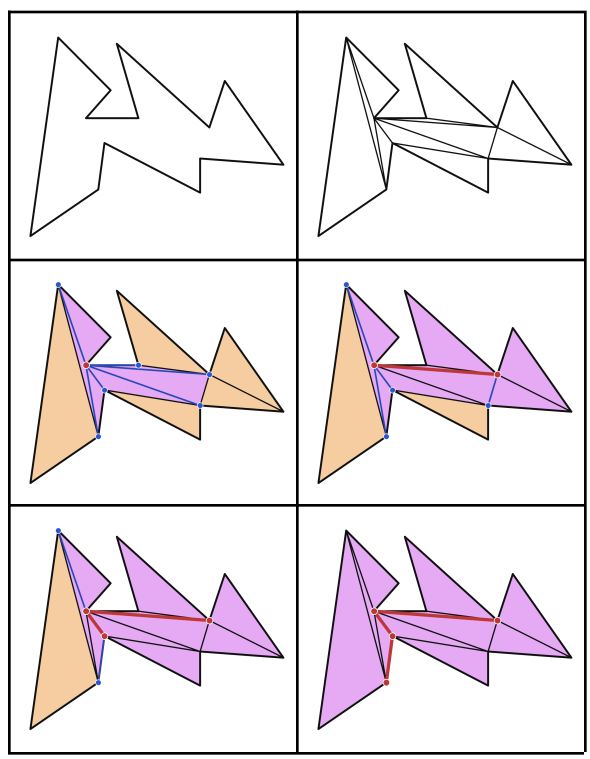

Figure 2: Step-by-step demonstration of the Triangulation Method, read left-right top-down.

Our greedy incremental construction operates using this point-graph. We choose the node with the most incident unseen faces as our initial vertex; another option could be to choose the node whose incident unseen faces together share the most area. Following the initial vertex, we only consider vertices that are connected to an already-chosen vertex by an edge. Of those, we select the best vertex by our metric, tiebreaking by the shortest edge length that would connect the new vertex to the existing network. The reasoning for the adjacency constraint is to ensure the final set of chosen edges are connected. Of course, any face incident to a chosen vertex is immediately removed from the "unseen" area; the construction ends when the unseen area is empty.

While this method is easy to implement and reason with, it is flawed because the difference between the "proven" and "true" seen areas is potentially large. It is possible for a triangulation to output a vertex that is incident to a relatively small number of triangles but whose real visibility is relatively large. While outputs are technically correct and meet our definition of validity, they do not meet our expectations for quality.

IV. Exact Visibility Method

Our second method is built on the first one, but is meant to be the main contribution of the project. Instead of using triangulation faces to prove visibility, we can compute the exact visibility polygons from points in the area. CGAL provides a function for this operation, `Simple_polygon_visibility_2()`, which works by sending rays out to every concave vertex. By using the vertices that are tangent to these rays along with the locations where the rays hit the edges of the polygon, you can create the visibility polygon. This was wrapped with a function to more easily work with CGAL's 2d polygon class. The overall visibility procedure takes $O(e)$ time, with e being the number of edges in the polygon.

With this utility function, we can improve the iterative construction to consider vertices directly. At each step, we examine all the unchosen vertices, generate their visibility polygons, and pick the candidate whose visibility polygon subtracts the most from the unseen area. CGAL provides a boolean set operations package, which we use for the polygon difference operation.

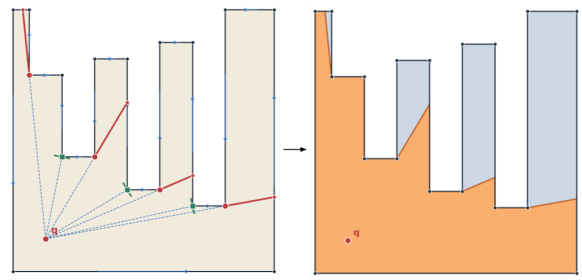

Figure 3: Visibility polygon construction. Left, rays cast from the concave vertices to the query point, with tangent rays highlighted. Right, the final visibility polygon.

After greedily selecting the ideal vertex, it must be connected to the existing path (to preserve connectivity of the patrol). This consists of a linear search through the already-chosen points to check for the minimum valid edge to the new vertex. A valid edge is any line segment for which no point on the line segment lies outside the polygon. This can be checked with the recursive procedure:

```
proc valid(segment, polygon) {
  for edge in polygon {
    pt = intersect(segment, edge)
    if pt == NULL {
      continue
    }
    if is_endpoint(pt, segment) {
      continue
    } else {
      seg1 = { pt, segment.p1 }
      seg2 = { pt, segment.p2 }
      return valid(seg1, polygon)
             && valid(seg2, polygon)
    }
  return polygon.contains(segment.mid)
}
```

since a segment that does not intersect anything lies wholly within a polygon if its midpoint lies inside the polygon, and a segment intersecting with a polygon boundary can be recursively split into subsegments that do not intersect with the polygon boundary. The complicated check is necessary because we consider edges that are collinear or tangent to elements of the polygon

boundary valid edges. Therefore, a simple intersect operation is not sufficient to determine whether or not an edge can contribute to a valid patrol route.

There is another issue with greedily selecting the vertex with maximum visibility at each iteration. While maximizing the visible area is a good metric, it is not perfect, since the *most* visible vertex is often located far away from the area of the polygon where the bulk of the patrol is likely to traverse. By choosing such a point, we may maximally reduce the unseen portion each step, but the extra cost in path length to reach those points is inefficient in the majority of cases. To avoid this issue, we simply discard all "convex" vertices of the polygon from consideration. The reasoning for this heuristic lies in the nature of non-convex shapes, whose concave corners are what blocks visibility at all. It is reasonable to expect that a guard seeking the shortest possible patrol will often choose concave corners anyway to minimize their traveling distance (a "rounding-corners" path).

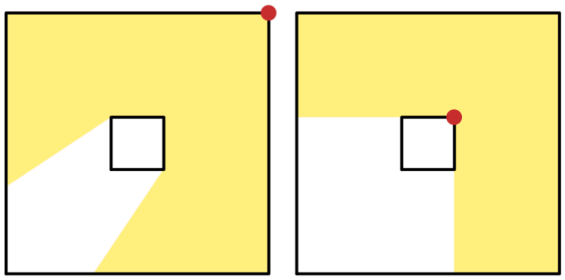

Figure 4: Example of a non-optimal vertex with optimal visibility (left) compared to an optimal vertex with non-optimal visibility (right).

V. Reduction Step

The aforementioned approaches are only concerned with producing a single connected patrol for $n = 1$ that covers our polygon. In order to handle $n > 1$ guards, we implement a greedy reduction algorithm to find which paths can be removed. Since visibility generation only comes from points, we are able to prune edges from any patrol without changing the total proven visibility. Since we are trying to minimize the total path length, we repeatedly remove the longest edge of the current graph until reaching n disjoint patrols.

For further patrol length optimization, we additionally consider redundant vertices in the patrol configuration and eliminate those as well. There are two basic cases of a potentially redundant point: it can either be a leaf of the graph formed by its polyline network patrol, or it can be a degree-2 node in that graph. In terms of visibility, a point `pt` in the patrol can be determined to be redundant by the procedure:

```
proc redundant(pt, patrols, polygon) {
  unseen = polygon
  for patrol in patrols {
    for vtx in patrol {
      if vtx == pt {
        continue
      }
      seen = visible(vtx, polygon)
      unseen -= seen
    }
  }
  return unseen.empty()
}
```

The primary new issue for pruning patrol vertices is preserving the connectivity of the patrol. Eliminating a one-degree node from a patrol has no effect on the rest of the patrol's connectivity, so there will be no connectivity issue if we remove a redundant one-degree node and its edge.

For second-degree nodes, we can resolve any break in connectivity by re-linking its two neighbor nodes directly. This simplifies one node (the redundant node) and its two edges into a single edge, which is guaranteed to either reduce or preserve the total patrol length. However, in this case it is important to check if the potential new edge formed between the neighbors remains wholly inside the polygon. We can use our intersection procedure from before to check for this. If the neighbors cannot be directly linked without leaving the polygon, we accept the redundant vertex as necessary to preserve connectivity.

Since there is no obvious method to resolve a redundant node of a higher degree, we do not consider them for pruning.

The goal of this point-pruning optimization is to cover cases where the patrol includes a vertex whose visibility polygon is itself completely

covered by other points in the configuration. In such a case, there is no need to make a "detour" from the main patrol to visit this point, and the guard should directly travel between another two vertices. We run this optimization after every reduction for a total of $O(hnk^2)$ additive cost, with h being the number of vertices in the solution patrol for $n = 1$ and k being the number of edges in the original polygon.

VI. Visualization

The animated visualizations were done in Python separately from the main program. For this, we use the Manim library. The visualization program takes as input the proposed polygon and the guards and points as a json. We re-wrote the visualization function in Python in order to save on having to call the C++ function every time and messing with a file to transition that data, and its underlying implementation is the same as the CGAL implementation, with a couple caveats to allow it to produce the right shapes. When the point sits on a vertex, the visibility polygon would "see" one side of that corner. We got around this by moving the point directly in the direction of the bisection of the interior polygon angle. This essentially moves the point away from the corner allowing it to see both sides of the walls that make it up. We then animate the guards moving across their paths, with the animations being longer or shorter depending on the number of guards.

Order-Notation Analysis

With k as the number of vertices/edges in the input polygon, Chin and Ntafos[1] describe an $O(k^4 log(k))$ algorithm to find the optimal solution for a single watchman, and Tan describes an $O(k)$ approximation for the same problem.

Our $n = 1$ solution using exact visibility runs in $O(hk^3)$ time to produce an approximation, with h being the number of vertices in the output patrol configuration. Our reduction step increases the order-notation to $O(hk^3 + hnk^2)$. Since our solution uses incremental construction and reduction, it makes sense that both terms are dependent on h. For future study, it may be interesting to look into output-dependent algorithms as a means to solve such problems.

Examples and Conclusion

In this section we will examine some of our results and see how our algorithm performed for both optimal and non-optimal cases.

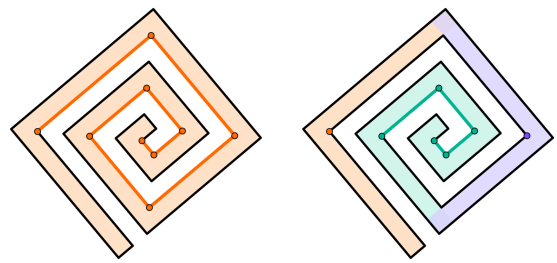

Figure 5: Left $n = 1$, right $n = 3$.

Figure 5 shows an optimal patrol configuration produced by our program. The algorithm correctly prunes the longest parts of the path when reducing to the $n > 1$ solution. We can see that not only are the longest segments trimmed, redundant points and edges are also pruned to shorten the single walker's path even further.

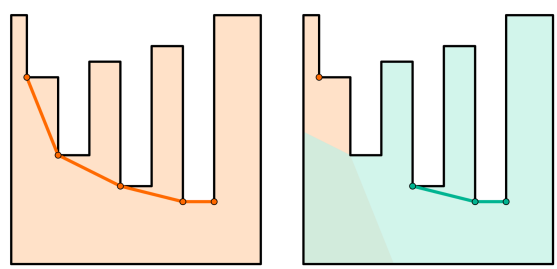

Figure 6: Left $n = 1$, right $n = 2$.

Figure 6 shows a non-optimal solution resulting from our algorithm's inability to consider arbitrary internal points in the polygon. While this is the shortest patrol configuration when only considering vertices of the original polygon, it is not the shortest possible patrol configuration.

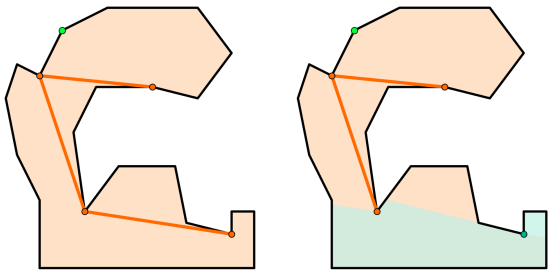

Figure 7: Left $n = 1$, right $n = 2$. More optimal vertex choice highlighted in green.

Figure 7 shows another non-optimal solution. For this polygon and patrol output, there is a shorter path if we choose the highlighted point instead. However, since we do not consider convex corners, it is not available to the algorithm as an option.

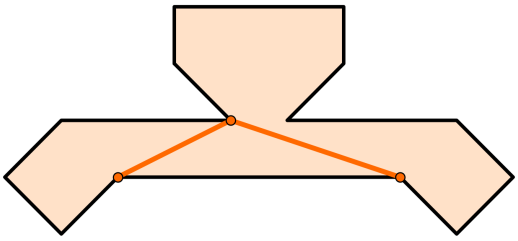

Figure 8: $n = 1$

Our final example, figure 8, shows that we choose an non-optimal path due to our lack of edge visibility. Our algorithm decides the upper vertex must be visited to see the entire area of the pentagonal protrusion, but a moving 360-degree camera traveling along the edge is able to see that entire area while traveling.

Conclusion and Future Work

Though there are many cases of our algorithm producing non-optimal solutions, this is something we expected. Due to the nature of our greedy algorithm, we hope that pruning out bad solutions can quickly lead us to find better ones, mirroring the nature of the more recent papers published where approximations are given for single and multi guard solutions.

Of course, there are many further improvements we could implement. The simplest would be to add edge visibility. Algorithms for it are already available and it would not significantly increase runtime. Considering convex vertices under certain criteria could also allow us to increase our search space to find better solutions that still lie on the polygons. Of course, being able to produce patrol vertices from any point within the polygon is ideal, although this would require a rethinking of our algorithm, perhaps to include half-plane operations as seen in Chin and Ntafos[1] 1991.

References

1. Chin WP, Ntafos S. Shortest watchman routes in simple polygons. *Discrete Comput Geom*. 1991;6:9-31. :10.1007/BF02574671
2. Tan X. Linear-time 2-approximation algorithm for the watchman route problem. In: *Cai JY, Cooper SB, Li A, eds. Theory and Applications of Models of Computation*. Vol. 3959. Berlin, Germany: Springer; 2006:181-191. :10.1007/11750321_17
3. Packer E. Computing multiple watchman routes. In: *McGeoch CC, ed. Experimental Algorithms*. Vol. 5038. Berlin, Germany: Springer; 2008:114-128. doi:10.1007/978-3-540-68552-4_9

This work is licensed under a Creative Commons Attribution 4.0 International License. All source code for the project is available under MIT license at the Github repository: https://github.com/pchen2215/n_watchmen_approximation